Lab 001: Beep Boop

Due Lecture 011/Nov 11th

Abstract

In this lab, you will be thrown feet-first into the ~~wonderful~~ world of logic design. Our goal for this lab is for you to familiarize yourself with your FPGA, SystemVerilog, and combinational logic. Your task is to design a basic ALU, which you will then validate in simulation.

Introduction

Hopefully by now, you have successfully downloaded Vivado. You can download the code and follow through the example from Lecture 010 to make sure everything is working properly and that you know how to use Vivado and the Basys3 board. Or you can raw dog it and figure it out as you go; up to you.

You will be implementing a rudimentary calculator that interacts with the world through your FPGA. We often refer to these circuits as "arithmetic logic units", or ALUs for short; ALUs serve as a fundamental building block for modern CPUs.

As you work your way through this first lab spec, you may encounter difficulties or points of confusion. Please consult the resources linked in the section below as well as Lecture 010, and don't hesitate to reach out to your leads for help (use dav-faq)!

Resources

Pinout

DAV Assignment 00: [Installing Vivado] 26-27

SystemVerilog Documentation 26-27

DAV LAB 1

(please spend some time consulting the internet first!)

1. The ALU

This module will contain the core functionality of your calculator: the **arithmetic logic unit**. After writing your logic, you will be testing your ALU with a testbench we created. Please try to keep the same naming conventions for the inputs and output for the simulations.

1.1 ALU logic

You will be designing an ALU that can perform two kinds of operations: **addition/subtraction**, and **arithmetic left/right shift**.

As inputs, your ALU will take:

- A **4-bit signal**, `op1`, for your first operand
- A **4-bit signal**, `op2`, for your second operand
- A **1-bit signal**, `operation`, to select between the two types of operation.
    - When `operation` is LOW, the ALU performs addition or subtraction (depending on the value of `sign`).
    - When `operation` is HIGH, the ALU performs an arithmetic left or right shift (depending on the value of `sign`).
- A **1-bit signal**, `sign`, to toggle the sign of the second operand.
    - When `sign` is LOW, perform the positive operation: either addition or a left shift.
    - When `sign` is HIGH, perform the negative operation: either subtraction or a right shift.

A total of ten input bits will correspond to the 16 switches on your FPGA

As output, your ALU will produce:

- A **20-bit signal** called `result` of the operation performed.

Before you start Veriloging, I would like to see if you could make a logical diagram first. Refer to Lecture 2 for what that may look like. Don't worry if this is wrong. It's just to get the juices flowing, and you will get to see the correct one at the end. **Please draw** the diagram and attach it when you reach the Checkoff section. (Note: you do not need to create gate level full adders and mux's; refer to the lecture and leave it at the logical level)

The output should be derived from a block of **combinational** logic in `miniALU.sv` with an `always_comb` construct, in which you will describe a combinational circuit. There's a commonly used hardware block (which we covered in lecture) to select between two or more values based on a boolean condition, which you'll need to properly assign your output.

1.2 ALU testing

With your module finished, it's time to test! The testbench for the miniALU can be found in the lab folder labeled miniALU_tb.sv. You can import the file by navigating through Project Manager -> Add Sources -> Add or create simulation sources -> finding your file and finishing.

To run the simulation, click on miniALU_tb in the Simulations Sources tab from the Sources and set it as Top. Then simply go to the Project Manager and click Run simulation in the Simulation section.

A whole bunch of things will show up, but don't worry about it. We will teach you how to write test benches and analyze all this, but for now, take a look at the Tcl Console. If all things worked correctly, it should display:

```
Time=0 op1= 7 op2= 5 operation=0 sign=0 result=     12
Time=20000 op1= 7 op2= 5 operation=0 sign=1 result=      2
Time=40000 op1= 7 op2= 5 operation=1 sign=0 result=    224
Time=60000 op1= 7 op2= 5 operation=1 sign=1 result=      0
```

Don't worry about the time, but as you can see our operands are 7 and 5 resulting in the following:
7+5 = 12, 7-5 = 2, 7 << 5 = 7 * 2^5 = 224, 7 >> 5 = 7 / 2^5 = 0
If your numbers are not matching, double check your logic.

1.3 ALU RTL Generation

Now that everything is working beautifully, we can use this logic on our FPGA. As seen in the lecture, there are a number of steps in getting from System Verilog to the FPGA. Feel free to look through every report and every design generated, but I want to highlight the logic schematic.

Set the miniALU.sv to be your Top. After running RTL analysis, you can open up the Elaborated Design where you can find the Schematic. There may be warnings, but if it looks good, it is probably fine. The schematic shows the datapath that explains how everything is getting processed. This is very helpful to understand what's really going on especially when it gets complicated (I can barely follow my game's logic from last year).

Looking at the diagram you may see some components that we went over during lecture. If you wanted, you could now even break this down to the gate level, but that's too much work. Take a screenshot and write a brief description of what's going on and explain anything different from your drawing. Bonus points if you explain why exactly the bit sizes are changing when they do.

2. CD (calculator display)

These modules will contain the logic that converts the result of your ALU to a human-readable format. Specifically, you'll be implementing an encoder for the 4 seven-segment displays on your FPGA. This time, you will be writing two modules called `displayEncoder.sv` and `sevenSegDigit.sv`.

2.1 displayEncoder.sv

You will now write a module to convert the result into the digits we will display. You will need to **compute the correct 4-bit number and connect them to your display driver** from the 20-bit result of your `miniALU`.

You'll notice that we are asking you to split a 20-bit signal into 4 4-bit signals. Think about why that is - note that we are converting from a binary (base 2) representation to a decimal (base 10) system.

As inputs, the decoder will take:

- A 20-bit signal result, representing the number computed by the `miniALU`.

As outputs, the decoder will produce:

- An array of four 4-bit signals, `numberBits [0:3]`, that drives the seven-segment displays on the FPGA.

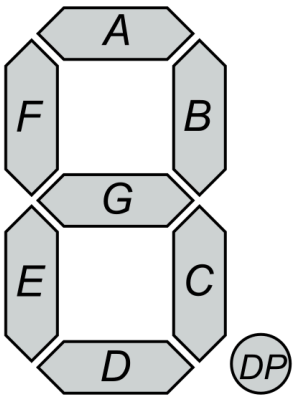
Logic-wise, think about what operators are available in Verilog; you'll need to perform some operation to extract each 4-bit `digit` from your 20-bit result.

Later, these `digit`s will be passed into four unique instances of `sevenSegDigit`, which will help you generate the array of 7-bit signals needed to drive the displays.

2.2 sevenSegDigit.sv

Each seven-segment display digit on the FPGA is driven by 8 bits. You will write a module to **drive 7 of these bits**.

The bit that drives each segment in each digit is shown in the visual to the right. The bits used to drive the display are listed in alphabetical order: the LSB (least significant bit), or bit 0, corresponds to segment A; bit 1 corresponds to segment B, and so on. The seventh bit corresponds to the G. Keep in mind that these segments are **active low**: a value of 0 turns each



segment on, while a value of 1 turns it off. (Note there is an 8th bit for DP, but we will not be using it)

As inputs, each display digit will take:

- A **4-bit signal**, digit, representing which numeral (0-9) should be displayed.

As output, each display digit will produce:

- A **7-bit signal**, displayBits, representing which segments on the seven-segment display should be on (remember the display is active LOW!).

The logic of this module should be fairly straightforward: there are ten possible digits as input, each mapped to a unique string of displayBits as output. In such situations, the case construct (similar to a switch-case statement in many coding languages) is useful: you will compare the input to each of the 10 digits and produce **the correct displayBits** if they match, or a **required "default" output** if none match. Refer to the resource document shown previously.

2.3 Testing the display

I won't be asking you to dive into this one, but take a look if you would like. Once again, it is test benching time. You can find display_tb in the folder, and give it a run from the steps from before. (Note make sure the previous simulation was closed)

This is what you should be getting (things out of order is not a big deal):

```
Time=0 result=   2345 displayBits = '{36,76,6,18}, {73, 153, 13, 37}, {36, 48, 25, 18}, {292, 304, 281, 274}
Time=20000 result=   6789 displayBits = '{4,0,15,32}, {9,1, 31, 64}, {2, 120, 0, 16}, {258, 376, 256, 272}
```

Note: Any of the sets displayed are considered correct and the order may be flipped. This is dependent on how you formatted your registers. These sets are hopefully exhaustive, but if you are confident in your implementation and it was not included, I may have overlooked it. You can ask one of the leads to review your code or continue on without checking.

2.4 seg_driver

Truth be told, the seven segment display is a little bit more complicated than this.

You can skip reading this if you want. The 4 displays are actually driven by one 8 bit signal and are instead controlled by 4 select pins enabling and disabling each segment. In order to display a number, you need to cycle through each individual display with a

high frequency clock, ~1-4k is optimal for human eyes. As we have yet to teach sequential logic and clocking, I have written it for you :).

The logic for this can be found in seg_driver.sv. It should be all ready to go. Just get it imported.

3. Putting everything together

Top module time. Refer to the lecture for details because I don't feel like writing it again >:).

3.1 calculator_top

With your modules complete, it's now time to connect them in the top module.
In `calculator_top`, instantiate your `miniALU`, `seg_driver`, `sevenSegDigit(s)` and `displayEncoder` modules. Create your board inputs (i.e. op1, op2, sign, operation) Create signals for the other inputs and outputs of each module, and connect them accordingly.

- The `result` output from `miniALU` should be connected to the `result` input to `displayEncoder`.
- Each of the `numberBits[0:3]` outputs from displayEncoder should be connected to a sevenSegDigit who outputs a part of displayBits[0:3]
- If you've been following this spec closely, you'll notice that an output doesn't exist yet :0.
    - You will need to include the following in your module declaration: input clk, output logic [6:0] segBits, output logic [3:0] trigger
    - As well as, the following declaration (Note: fill in the #):
      seg_driver DUT# (.displayBits(displayBits), .clk(clk), .trigger(trigger), .segBits(segBits));

3.2 A plan for your pins

Once that's complete, set calculator_top to be your top module and synthesize (Note: does take some time). You will now be able to open the Synthesized Design or write your constraint by hand for some reason. You can now go to Window -> I/O Ports if it is not already there and begin assigning things. Refer to the website at the beginning of the document for pin #, and use I/O Standard -> LVCMOS33.

- Set CLK to the system clock
- segBits should be connected to seven segment A-G
- Trigger connected to the following: U2, U4, V4, W4. (Note: you may need to reverse the order if your numbers come out weird i.e. thousand -> 0001)

- The op1 input to miniALU should be assigned to the first four (15-12) switches.
- The operation input to miniALU should be assigned to the next (11) switch.
- The sign input to miniALU should be assigned to the last (10) switch.
- The op2 input to miniALU should be assigned to the next four (9-6) switches.

Afterwards, give it a save and your constraint file should be generated

3.3 I am all done!!!

That's enough verification and simulations. It's time to test it in the real world! Set your top module calculator_top.sv as the top module in the file hierarchy, and generate a bitstream.

Once generated, connect your board, open the hardware manager, open target, detect the board, and program the device. (Refer to lecture) Make sure to turn on your board with the physical switch. I have spent way too long staring at my board to work when it was not powered on >:(.

3.4 Calculator RTL Generation

Just kidding, I want to give you more work. Once again, take a look at the logical schematic and do a quick trace through the logic of the calculator as a whole. Include one last BRIEF description.

4. Check off

Congrats you made it. To get checked off, hunt one of the leads in person to show us your amazing calculator or send a video in your team channel with the following test cases.
10 + 15 = 25
3 - 2 = 1
1 << 8 = 256
12 >> 2 = 3
Be sure to show us that document with all your logic schematics as well (initial draft, miniALU, calculator) and give a BRIEF description of what's going on in each.